EagleEyes Parking

Maintenance Plan & Manual · CSCE Capstone, Group 3 · University of North Texas

1. Introduction

Application name: EagleEyes Parking (EagleEyes)

Brief description: EagleEyes is a mobile and web application that helps UNT students, faculty, and staff find open parking on campus in real time. It shows a live campus map with lots color-coded by occupancy, lets users filter lots by the permit they hold and by distance, and hands off directions to the phone's maps app. Commuters can report parking hazards with photos, and administrators review those reports, post campus alerts and events, and manage other administrators. This manual explains how to install, run, maintain, troubleshoot, and update EagleEyes after deployment so that another developer or system administrator can keep it running.

Technologies used:

- **Frontend:** React Native with Expo SDK 54 (Expo Router), TypeScript/JavaScript; react-native-maps (mobile) and Google Maps via @vis.gl/react-google-maps (web); expo-image-picker; react-native-calendars.
- **Backend:** Python 3.9+, FastAPI and Uvicorn; SQLAlchemy ORM with Alembic; JWT authentication with python-jose and passlib/bcrypt; httpx for ParkZen API calls; Pillow / pillow-heif for photo conversion.
- **Services:** Neon serverless PostgreSQL (database), ParkZen API (live lot occupancy), Google Maps, Gmail SMTP (password-reset emails).
- **Project tools:** Git and GitHub (source control, pull requests), Trello (backlog and bug tracking).

Team members: Nhan Vu (Team Lead), Thuan (Emma) Nguyen, Aaditya Yadav, Alperen Dalkir, Luis Lozano.

2. Installation and Running the Application

EagleEyes has two parts: a FastAPI backend (Python) for accounts, the database and live ParkZen data, and an Expo / React Native frontend for iOS, Android and the web. The backend must be running before the frontend can show data. End users of the web version do not install anything (see User Manual, Section 3).

2.1 Software Requirements

Software	Version	Purpose
Git	Any recent	Download the code
Python	3.9 or newer	Backend
Node.js and npm	20.19.4 or newer	Frontend (required by Expo SDK 54)
Xcode / Android Studio	Latest	iOS simulator / Android emulator (optional)
Expo Go app	Latest	Testing on a phone (optional)

External services: Neon PostgreSQL (database), ParkZen API (lot occupancy), Google Maps (map display), and Gmail SMTP (password-reset emails, optional). Credentials come from the project lead.

2.2 Installation Steps

All commands start from the repository root (`Eagle-Eye-Parking/`) unless stated otherwise.

Step 1 – Clone the repository:

```
git clone https://github.com/ReyonaD/Eagle-Eye-Parking.git
cd Eagle-Eye-Parking
```

Step 2 – Install the backend (terminal 1):

```
cd EagleEyes/backend
python3 -m venv venv
source venv/bin/activate        # Windows: venv\Scripts\activate
pip install -r requirements.txt
```

Step 3 – Install the frontend (terminal 2, from the repository root):

```
cd EagleEyes/frontend
npm install
```

2.3 Configuration

EagleEyes requires two `.env` files containing private credentials. Git ignores these files. Never commit them or put their real values in documents.

Backend configuration (`EagleEyes/backend/.env`):

Variable	Required	Purpose
DATABASE_URL	Yes	Neon connection string, e.g. postgresql://<user>:<password>@<host>/<db>?sslmode=require
SECRET_KEY	Yes	Signs login tokens. Use a long random string.
PARKZEN_USERNAME PARKZEN_PASSWORD	Yes	ParkZen account for live lot data

Optional settings: SMTP_USER and SMTP_PASSWORD configure Gmail password-reset emails. If omitted, reset codes are printed in the backend terminal. Other optional settings with defaults include ALGORITHM, ACCESS_TOKEN_EXPIRE_MINUTES, PARKZEN_CLIENT_ID, SMTP_HOST, SMTP_PORT, and EMAIL_FROM_NAME.

Frontend configuration (`EagleEyes/frontend/.env`): Set EXPO_PUBLIC_API_URL (required) to the backend address: http://localhost:8000 for web or iOS simulator, http://10.0.2.2:8000 for the Android emulator, or http://<computer-LAN-IP>:8000 for a physical phone on the same Wi-Fi. Set EXPO_PUBLIC_GOOGLE_MAPS_API_KEY (required for web maps) to enable Google Maps on the web version.

Database: No manual setup is needed. Every time the backend starts, it automatically creates missing tables and adds missing columns.

First administrator: New accounts are commuters by default. To create the first administrator, register an account, then have an authorized team member with access to the Neon database run the following command in the Neon SQL editor:

```
UPDATE users SET role='admin', is_owner=TRUE WHERE email='<email>';
```

Log out and log back in for the new administrator role to take effect.

2.4 Running the Application

After installing the dependencies and configuring the `.env` files (Section 2.3), start the backend and frontend in separate terminals.

Backend (from `EagleEyes/backend`):

```
source venv/bin/activate
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Frontend (from `EagleEyes/frontend`):

```
npx expo start --clear
```

Open http://localhost:8000/docs to verify the backend is running. For the frontend, press **w** to open the web version, or scan the QR code with Expo Go on a phone.

3. Maintenance

This section describes the routine tasks needed to keep EagleEyes running smoothly.

3.1 Routine Maintenance Tasks

Maintenance Task	Frequency	Procedure
Verify parking information	Quarterly and when discrepancies are reported	Review lot names, permit types, and occupancy information for mismatches between EagleEyes and official UNT parking information. Correct verified discrepancies and update the application when necessary.
Check application functionality	Weekly	Test the interactive map, lot details, permit filtering, nearby parking, hazard reporting, and administrative features to confirm they work as expected.
Review application logs	Weekly	Check frontend, backend, and server logs (including diagnostic logs submitted from Help & Support) for errors, failed requests, and unexpected behavior. Investigate recurring errors and document the findings.
Monitor application performance	Weekly	Check map loading speed, API response time, and responsiveness. Investigate slow loading, failed requests, UI crashes, or excessive resource usage.
Update software dependencies	Monthly and when needed	Review frontend and backend dependencies. Test compatible updates in a development environment before applying them to production.
Apply security updates	As soon as practical after evaluation	Review security advisories for the operating system, runtime, libraries, and hosting environment. Apply necessary patches and verify that the application still works correctly.
Review access permissions	Monthly	Verify that administrative functions are accessible only to authorized users. Remove unnecessary permissions and review account access when team responsibilities change.

3.2 Maintenance Procedures

Step 1: Check application availability. Open EagleEyes and confirm that the application loads successfully. Verify that the map displays correctly and that parking lot details can be opened.

Step 2: Test important features. Test parking lot selection, permit filtering, nearby parking, hazard reporting, and relevant administrative functions. Record any failures for further investigation.

Step 3: Review errors and performance. Inspect the available application and server logs. Look for repeated errors, failed API requests, slow responses, and other signs of performance problems.

Step 4: Maintain application data. Verify that parking lot information and occupancy data retrieved from the ParkZen API load correctly and display as expected. If data is missing, outdated, or inconsistent, investigate the API integration and report persistent issues to the API provider. Separately, verify that hazard reporting and event management work correctly: submitting, reviewing, and approving hazard reports, and adding and displaying campus events. Ensure that reported hazards go through the administrative review process and that events appear correctly in the application.

Step 5: Update and test dependencies. Review available updates, check for security concerns, and apply approved changes in a development or staging environment before deploying them to production.

Step 6: Document the maintenance work. Record the date, task performed, issues discovered, changes made, and test results. Keep maintenance records available for future developers and administrators.

3.3 Maintenance Precautions

Before changing the production application, the maintenance team should review the proposed changes and back up important data when applicable. Updates should be tested before deployment, and the team should have a recovery plan in case an update causes problems. Administrative access and sensitive configuration values must be protected.

Because EagleEyes provides parking information, hazard reporting, and event planning for UNT commuters, maintaining accurate information, reliable map functionality, and secure administrative access is especially important.

4. Troubleshooting

4.1 User Cannot Log In

Problem: A user receives an error message when trying to log in to EagleEyes.

Possible causes:

- Incorrect email or password
- Internet connection problems
- Authentication server or database unreachable

Troubleshooting steps:

1. Verify that the user entered the correct login credentials (the user can reset a forgotten password from the login screen).
2. Check that the device has an active internet connection.
3. Verify that the FastAPI backend server is running (http://<backend>:8000/docs loads).
4. Check the backend logs for authentication errors from the `/login` endpoint.
5. Verify that the PostgreSQL (Neon) database is connected and functioning correctly.

4.2 Parking Availability Not Loading

Problem: The application does not display parking availability or shows outdated parking information.

Possible causes:

- The ParkZen API is unavailable
- The backend server cannot retrieve parking information
- The application is experiencing network issues

Troubleshooting steps:

1. Confirm the device is connected to the internet.
2. Close and reopen the EagleEyes application.
3. Check the backend `/api/lots` endpoint to verify that parking information is being returned.
4. Review the backend logs for errors involving the ParkZen API.
5. Verify that the ParkZen credentials and configuration in `backend/.env` are correct.
6. If necessary, restart the backend after resolving the issue.

4.3 Incorrect Parking Permit Information

Problem: A parking lot displays an incorrect permit type or the wrong permit category.

Possible causes:

- Permit information received from ParkZen is incorrect or incomplete
- Permit classification logic is not grouping permits correctly

Troubleshooting steps:

1. Identify the parking lot displaying incorrect information.
2. Review the permit information returned by the ParkZen API (`/api/permits` and `/api/lots`).

3 Check the application's permit classification logic to ensure permits are categorized correctly.

4 Correct any classification errors and test the affected parking lots before deploying the update.

5. Future Maintenance Updates

5.1 Bug Tracking and Fixing

Bugs and maintenance tasks are tracked through the team's GitHub repository and Trello board. Problems are found in three ways: user feedback, the routine maintenance checks described in Section 3, and the app's built-in diagnostic reporting. From the Help & Support screen, a commuter can send a snapshot of the app's recent console output along with their device information, so the team can trace bugs that only happen on a specific device (for example, a map issue on one Android version) without needing the device in hand. Each problem is recorded as a task with a short description, steps to reproduce it, and a priority. Problems that affect core functionality such as login, the map, or live occupancy data are high priority and are worked on first.

Fixes follow the same workflow the team uses for development. A developer creates a personal branch from `main`, implements and tests the fix locally using the steps in Section 2, and opens a pull request on GitHub. Another team member reviews the pull request before it is merged; after merging, the fix is verified in the running application and added to the maintenance log (Section 3.2, Step 6). Larger changes that affect requirements go through the team's change request process, where a change request form is reviewed and approved before implementation. CR-001 (the occupancy color update) and CR-002 (the Note to Admin feature) were both handled this way.

5.2 Planned Improvements

Several improvements are planned for future sprints:

- **Push notifications,** so users receive campus alerts and closure announcements even when the app is closed, instead of only seeing them in the in-app Alerts feed.
- **Enabling the ParkZen mobile SDK** in the app (currently disabled in development builds) so users can contribute anonymous parking data and improve occupancy accuracy.
- **Server-side caching** of ParkZen lot responses to reduce external API calls and keep response times reasonable as the number of users grows.
- **Expanded admin tools:** building on the existing dashboard, note history, and diagnostic logs with usage statistics and reporting for UNT Parking Services.

References

[1] EagleEyes Group 3. *EagleEyes – User Manual.* GitHub repository, docs/USER_MANUAL.md. https://github.com/ReyonaD/Eagle-Eye-Parking

[2] EagleEyes Group 3. *Software Requirements Specification (Sprint 1)* and *Design Documents 1 & 2.* Repository docs/01-Requirements and docs/02-Design.

[3] EagleEyes Group 3. *Change Request Forms CR-001 and CR-002.* Team project records.

[4] EagleEyes Group 3. Trello board (sprint backlog and bug tracking). https://trello.com/b/IFMGacCF

[5] Expo. *Expo Documentation (SDK 54).* https://docs.expo.dev

[6] FastAPI. *FastAPI Documentation.* https://fastapi.tiangolo.com

[7] Neon. *Neon Serverless Postgres Documentation.* https://neon.tech/docs